## Using the Database in Practice

We know that, having configured our Rails app to talk to our Postgres database, our Rails app started successfully; however, we're relying on the *absence of an error* as proof that the database is connected correctly. While, technically, that's all we need, let's make sure it's working as expected by interacting with the database from our app. This will also give us more practice at developing our Rails app with Docker, via the Compose CLI.

Let's generate a basic UsersController in our Rails app. For the sake of speed, we're going to just use Rail's generate scaffold command:

```
$ docker-compose exec web \
    bin/rails g scaffold User first_name:string last_name:string
      invoke  active_record
      create    db/migrate/20190115100954_create_users.rb
      create    app/models/user.rb
      invoke  resource_route
       route    resources :users
      invoke  scaffold_controller
      create    app/controllers/users_controller.rb
      invoke    erb
      create      app/views/users
      create      app/views/users/index.html.erb
      create      app/views/users/edit.html.erb
      create      app/views/users/show.html.erb
      create      app/views/users/new.html.erb
      «...»
```

**Linux Users: File Ownership**



Remember to chown the files we've just generated by running the following command:

```
$ sudo chown <your_user>:<your_group> -R .
```

See File Ownership and Permissions, on page 199, for more details.

Now we need to run the migrations to create the Users table. You should be starting to get comfortable with running standard Rails commands targeted against our web service using Compose, just like this:

```
$ docker-compose exec web bin/rails db:migrate
== 20190115100954 CreateUsers: migrating ==================================
-- create_table(:users)
   -> 0.0585s
== 20190115100954 CreateUsers: migrated (0.0587s) =========================
```